부록 · Docker 샌드박스 · Compose

Docker Sandboxes 완전 초보 실전 구축 교본

Linux 가상환경 + Codex AI + SSH + Kotlin/Spring Boot + MariaDB + 외부접속 + 영구저장

Docker Sandboxes 0.43.0 계열 기준 · sbxenv.yaml 실전 확장판

이 교본은 Docker Sandboxes의 `sbx`와 실험적 `sbx env`/`sbxenv.yaml`을 이용하여 AI 코딩 환경을 설계하는 방법을 초보자 관점에서 설명함. Docker Compose와 sbxenv.yaml의 역할을 구분하고, 실제 외부 접속과 영구 보존에서 자주 생기는 오해를 바로잡는 데 중점을 둠.

이 문서에서 고친 곳

- 문서 생성 과정의 대화 흔적(「요청한」)을 지웠습니다.

0. 가장 중요한 결론부터

- sbxenv.yaml은 docker-compose.yml이 아님. `services: mariadb:`처럼 여러 컨테이너 서비스를 선언하는 파일이 아님.
- `ports`는 포트를 연결할 뿐 SSH 서버, MariaDB 서버, Spring Boot를 자동 설치하거나 실행하지 않음.
- `lifecycle`은 Sandbox 내부가 아니라 호스트에서 실행되는 명령이므로 Sandbox 내부 apt 설치용으로 사용하면 안 됨.
- 로컬 개발에서는 `workspace: ./workspace`가 호스트 디렉터리를 Sandbox에 연결하므로 프로젝트 소스 영구 보존의 핵심임.
- Cloud persistent volume은 cloud-only이며 Sandbox 종료 시 snapshot으로 저장됨.
- SSH와 MariaDB를 LAN에 공개할 때는 `hostIP: 0.0.0.0`뿐 아니라 호스트 방화벽, 서비스 listen 주소, 계정 보안까지 함께 설정해야 함.
- Codex 자격증명은 YAML에 API 키를 평문으로 넣기보다 Docker Sandboxes Secret/OAuth 기능을 우선 사용함.

1. 목표 아키텍처

```text
외부 Windows / Linux PC
          |
          | LAN
          ▼
Docker Host (예: SERVER_IP)
          |
          ├── TCP 2222 ──────> Sandbox TCP 22
          |                     SSH Server
          |
          ├── TCP 8080 ──────> Sandbox TCP 8080
          |                     Kotlin + Spring Boot
          |
          └── TCP 3307 ──────> Sandbox TCP 3306
                                MariaDB

Sandbox 내부 개발 흐름
Codex AI
   |
   ├── Kotlin 코드 분석/수정
   ├── Gradle clean/build/test
   └── Spring Boot 실행
            |
            ▼
        JPA / JDBC
            |
            ▼
         MariaDB
```

이 구조의 핵심은 'Sandbox 자체'와 '그 안에서 실행되는 서비스'를 구분하는 것임. sbxenv.yaml은 Sandbox의 agent, workspace, 포트, 자원, 환경변수 등을 선언함. SSH/MariaDB/JDK 같은 내부 도구를 반복 가능하게 준비하려면 적절한 Sandbox template 또는 kit을 만들거나, 생성 후 `sbx exec`로 내부 설치 과정을 수행해야 함.

2. 권장 프로젝트 디렉터리

```text
wiz-sandbox/
├── sbxenv.yaml              # Sandbox 환경 선언
├── workspace/               # 영구 보존되는 주 작업 폴더
│   ├── build.gradle.kts
│   ├── settings.gradle.kts
│   ├── gradlew
│   ├── gradle/
│   └── src/
├── docs/                    # Codex 참고문서(read-only 권장)
├── backups/                 # 삭제 전 백업
├── config/
│   ├── ssh/
│   ├── mariadb/
│   └── spring/
└── scripts/
    ├── sandbox-install.sh
    ├── sandbox-start.sh
    ├── db-init.sql
    ├── backup.sh
    └── check.sh
```

환경 파일은 직접 마운트되는 workspace 바깥에 두는 것이 권장됨. 위 구조에서는 `sbxenv.yaml`과 `workspace/`가 형제 디렉터리이므로 관리하기 편함.

3. sbxenv.yaml 완전체 기준안

```yaml
schemaVersion: "1"

name: wiz-full-dev
agent: codex

workspace: ./workspace

additionalWorkspaces:
  - path: ./docs
    readOnly: true

args:
  profile:
    default: dev
    description: Spring Boot profile
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}

  DB_HOST: 127.0.0.1
  DB_PORT: "3306"
  DB_NAME: wizdb
  DB_USER: devuser

  JAVA_TOOL_OPTIONS: "-Xms512m -Xmx2g"

sandboxOptions:
  cpus: 4
  memory: 8g

ports:
  # SSH
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4

  # Spring Boot
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4

  # MariaDB
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4

lifecycle:
  initialize:
```

```
    - name: Prepare host directories
      command: |
        mkdir -p workspace
        mkdir -p docs
        mkdir -p backups
        mkdir -p config/ssh
        mkdir -p config/mariadb
        mkdir -p config/spring
        mkdir -p scripts
      timeout: 1m

  preRemove:
    - name: Backup workspace before removal
      command: |
        mkdir -p backups
        tar -czf backups/wiz-workspace-backup.tar.gz workspace
      timeout: 5m
```

이 파일은 Sandbox의 외형과 연결을 선언하는 기준안임. 이 YAML만으로 Sandbox 내부에 sshd와 MariaDB가 설치되는 것은 아님. 다음 장에서 내부 서비스 준비를 별도로 설명함.

4. YAML 항목 한 줄씩 이해하기

항목	설명
schemaVersion	현재 환경 파일 스키마 버전. 지원되지 않는 필드는 loader가 거부할 수 있음.
name	Sandbox 이름. `--name`으로 실행 시 일시적으로 override 가능.
agent: codex	Sandbox에서 사용할 AI coding agent를 Codex로 선택.
workspace	주 프로젝트 디렉터리를 Sandbox workspace로 mount.
additionalWorkspaces	추가 코드/문서 디렉터리를 mount.
readOnly: true	Codex가 참고는 가능하지만 해당 추가 workspace 수정은 막음.
args.profile	dev/test/prod 중 실행 시 선택 가능한 사용자 인자.
env	Sandbox agent 세션에 전달할 환경변수.
sandboxOptions.cpus	Sandbox CPU 수.
sandboxOptions.memory	Sandbox 메모리 제한.
ports.sandbox	Sandbox 내부 서비스 포트.
ports.host	Docker Host에서 사용할 포트.
ports.hostIP	0.0.0.0이면 호스트의 IPv4 인터페이스에 게시. LAN 공개 효과가 있음.
protocol	tcp4로 IPv4 TCP 게시.
lifecycle.initialize	Sandbox 내부가 아닌 호스트에서 환경 적용 전에 실행되는 명령.
lifecycle.preRemove	Sandbox 제거 전에 호스트에서 실행되는 명령.

5. CPU 4 / RAM 8GB

```yaml
sandboxOptions:
  cpus: 4
  memory: 8g
```

Spring Boot, Gradle, Codex, MariaDB를 한 Sandbox에서 학습/개발 목적으로 함께 사용한다면 4 CPU와 8GB는 무난한 출발점임. 대형 프로젝트 빌드나 많은 테스트가 있으면 메모리를 늘릴 수 있음. 환경 파일에서 자원 설정을 변경해도 기존 Sandbox에 즉시 적용되지 않을 수 있으므로 제거 후 재생성이 필요할 수 있음.

6. Codex AI 기본 설정

Codex는 `agent: codex`로 지정함. OpenAI 자격증명은 YAML에 키를 직접 적는 방식보다 Docker Sandboxes Secret/OAuth를 우선 사용함.

```bash
sbx secret set openai --oauth
sbx secret ls

cd wiz-sandbox
sbx env plan
sbx env run
```

OAuth 방식은 OpenAI Secret을 저장하고 Sandbox proxy가 agent의 API 요청을 인증하는 구조로 사용할 수 있음. Secret 값 자체를 프로젝트 YAML에 넣지 않는 것이 핵심임.

7. Codex에게 실제로 시킬 작업 20개

1. 현재 Kotlin + Spring Boot 프로젝트의 전체 폴더 구조를 분석하고 개선점을 설명해줘.
2. build.gradle.kts를 확인하고 Spring Web, JPA, MariaDB 의존성이 올바른지 점검해줘.
3. Member Entity와 Repository를 만들어줘.
4. Service 계층을 추가하고 Controller가 Repository를 직접 호출하지 않도록 수정해줘.
5. REST API의 요청/응답 DTO를 추가해줘.
6. MariaDB 연결 설정을 application.yml과 profile별 파일로 분리해줘.
7. 개발 profile에서 Hibernate ddl-auto=update가 적용되도록 해줘.
8. 운영 profile에서는 ddl-auto를 안전한 값으로 변경해줘.
9. GET /api/members API를 만들고 테스트 코드를 작성해줘.
10. POST /api/members 입력 검증을 추가해줘.
11. 예외 처리를 @RestControllerAdvice 방식으로 정리해줘.
12. 현재 프로젝트를 ./gradlew clean build로 검증해줘.
13. 빌드 오류가 있으면 원인을 분석하고 수정한 뒤 다시 build해줘.
14. Spring Boot가 8080에서 정상 기동되는지 확인해줘.
15. MariaDB 연결 오류가 있으면 host, port, username 설정을 점검해줘.
16. JPA SQL 로그가 개발 profile에서만 보이게 해줘.

17. 프로젝트 README에 실행 방법을 초보자용으로 작성해줘.

18. 보안상 소스에 하드코딩된 비밀번호가 있는지 찾아줘.

19. Git에 커밋하면 안 되는 파일을 찾아 .gitignore를 보완해줘.

20. 수정이 끝나면 변경 파일 목록과 실행 명령을 요약해줘.

8. SSH 2222 → 22

```yaml
ports:
  - sandbox: 22
    host: 2222
    hostIP: 0.0.0.0
    protocol: tcp4
```

이 설정은 '포트 통로'만 만듦. Sandbox 내부 TCP 22에서 실제 sshd가 listen 중이어야 함. 또한 Sandbox 내부 사용자의 비밀번호 또는 SSH public key 인증이 준비되어 있어야 함.

9. SSH 서버 준비 개념

반복 가능한 환경에서는 SSH 서버가 포함된 custom template/kit을 만드는 것이 이상적임. 학습 단계에서는 Sandbox 생성 후 root 권한으로 내부 패키지를 설치하는 방식도 가능함. 실제 명령은 template의 Linux 배포판에 따라 달라짐.

```bash
sbx env create
sbx exec -it -u root wiz-full-dev bash

# Sandbox 내부가 Debian/Ubuntu 계열인 경우의 예
apt-get update
apt-get install -y openssh-server
mkdir -p /run/sshd

# sshd 설정과 사용자/키 구성 후
/usr/sbin/sshd
```

systemd가 없는 이미지에서는 `systemctl start ssh`가 동작하지 않을 수 있으므로 `/usr/sbin/sshd` 직접 실행 또는 이미지의 init 방식에 맞춘 실행이 필요함.

10. SSH 접속 방법

```bash
# Docker Host 자체에서
ssh -p 2222 devuser@127.0.0.1

# LAN 다른 PC에서
ssh -p 2222 devuser@SERVER_IP
```

SERVER_IP은 예시 Docker Host IP임. 실제 Host IP를 사용해야 함.

11. Windows에서 SSH 포트 검사

```powershell
Test-NetConnection SERVER_IP -Port 2222
```

TcpTestSucceeded가 True이면 Host까지 포트 연결은 가능하다는 뜻임. SSH 로그인 실패는 그 다음 단계인 sshd, 사용자, 키/비밀번호, 방화벽 설정을 확인함.

12. Spring Boot 8080 → 8080

```yaml
ports:
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4
```

Spring Boot가 Sandbox 내부에서 127.0.0.1에만 bind되어 있으면 외부 게시가 기대대로 동작하지 않을 수 있음.
개발용으로는 서버 주소를 적절히 설정하고 `ss -lntp`로 실제 listen 상태를 확인함.

```bash
sbx env exec -- ./gradlew bootRun
sbx env exec -- bash -lc "ss -lntp | grep 8080 || true"
```

13. Spring Boot 기본 프로젝트 구성

```text
workspace/
├── build.gradle.kts
├── settings.gradle.kts
├── gradlew
├── gradle/
└── src/
    ├── main/
    │   ├── kotlin/com/wiz/app/
    │   │   ├── Application.kt
    │   │   ├── controller/
    │   │   ├── service/
    │   │   ├── repository/
    │   │   ├── entity/
    │   │   └── dto/
    │   └── resources/
    │       ├── application.yml
    │       ├── application-dev.yml
    │       ├── application-test.yml
    │       └── application-prod.yml
    └── test/
```

14. build.gradle.kts 기본 예

```kotlin
plugins {
    kotlin("jvm") version "2.2.20"
    kotlin("plugin.spring") version "2.2.20"
    kotlin("plugin.jpa") version "2.2.20"
    id("org.springframework.boot") version "3.5.6"
    id("io.spring.dependency-management") version "1.1.7"
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

15. Spring Profile dev/test/prod

```bash
args:
  profile:
    default: dev
    enum:
      - dev
      - test
      - prod

env:
  SPRING_PROFILES_ACTIVE: ${{ env.args.profile }}
# 개발
sbx env run --env-arg profile=dev

# 테스트용 별도 Sandbox 이름
sbx env create --name wiz-test --env-arg profile=test
sbx env exec --name wiz-test -- ./gradlew test

# 운영 profile 검증
sbx env create --name wiz-prod-check --env-arg profile=prod
```

16. application.yml

```yaml
server:
  port: 8080

spring:
  application:
    name: wiz-sandbox-app
```

17. application-dev.yml

```yaml
spring:
  datasource:
    url: jdbc:mariadb://${DB_HOST:127.0.0.1}:${DB_PORT:3306}/${DB_NAME:wizdb}
    username: ${DB_USER:devuser}
    password: ${DB_PASSWORD:}

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

DB_PASSWORD를 소스에 고정값으로 넣지 않는 것이 중요함. 애플리케이션 자체 DB 비밀번호 전달 전략은 별도로 설계해야 하며, Docker Sandboxes의 agent service secret과 애플리케이션 환경변수는 같은 개념이 아님.

18. MariaDB 3307 → 3306

```yaml
ports:
  - sandbox: 3306
    host: 3307
    hostIP: 0.0.0.0
    protocol: tcp4
```

이 역시 포트 게시만 수행함. MariaDB 서버가 Sandbox 내부 3306에서 실행되어야 하며, 외부 접속을 허용하는 bind-address와 사용자 host 권한도 별도로 확인해야 함.

19. MariaDB 설치 예

```bash
sbx exec -it -u root wiz-full-dev bash

# Debian/Ubuntu 계열 예
apt-get update
apt-get install -y mariadb-server mariadb-client

# 이미지에 systemd가 없다면 mariadbd를 이미지 방식에 맞게 실행
mariadbd --version
```

20. MariaDB 초기화 SQL 예

```sql
CREATE DATABASE IF NOT EXISTS wizdb
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'devuser'@'%'
  IDENTIFIED BY 'CHANGE_ME_STRONG_PASSWORD';

GRANT ALL PRIVILEGES ON wizdb.*
  TO 'devuser'@'%';

FLUSH PRIVILEGES;
```

문서의 비밀번호는 예시 placeholder임. 실제 비밀번호는 별도 Secret/보안 저장소에서 관리하고 Git에 커밋하지 않음.

21. MariaDB 외부 접속

```bash
# Docker Host
mariadb -h 127.0.0.1 -P 3307 -u devuser -p

# LAN 다른 PC
mariadb -h SERVER_IP -P 3307 -u devuser -p
```

22. Windows 포트 점검

```powershell
Test-NetConnection SERVER_IP -Port 2222
Test-NetConnection SERVER_IP -Port 8080
Test-NetConnection SERVER_IP -Port 3307
```

Host 포트	Sandbox 포트	서비스	접속 예
2222	22	SSH	ssh -p 2222 devuser@HOST_IP
8080	8080	Spring Boot	http://HOST_IP:8080
3307	3306	MariaDB	mariadb -h HOST_IP -P 3307 -u devuser -p

23. hostIP: 0.0.0.0의 의미

hostIP를 생략하면 Docker Sandboxes의 로컬 포트 게시 기본은 loopback 중심임. LAN의 다른 장치에서 접근시키려면 환경 파일에서 명시적으로 hostIP를 설정할 수 있음. 0.0.0.0은 편리하지만 노출 범위가 넓어지므로 SSH와 DB는 특히 주의해야 함.

인터넷 공유기에서 추가 포트포워딩까지 하면 공인 인터넷에 노출될 수 있으므로 개발환경의 SSH/MariaDB 직접 공개는 권장하지 않음.

24. 영구 workspace

```yaml
workspace: ./workspace
```

로컬 Sandbox에서 가장 중요한 영구 보존 수단임. 프로젝트 파일을 Sandbox 내부 임의 디렉터리에만 저장하지 말고 workspace에 두면 Sandbox가 없어져도 호스트의 `./workspace` 파일은 유지됨.

- Spring Boot 소스코드
- Gradle 설정
- SQL 초기화 파일
- README
- 테스트 코드
- 프로젝트별 문서

25. 추가 read-only workspace

```yaml
additionalWorkspaces:
  - path: ./docs
    readOnly: true
```

법률 문서, 설계서, API 명세, 회사 규칙처럼 Codex가 참고는 해야 하지만 수정하면 안 되는 자료를 넣기에 적합함.

26. 삭제 전 자동 백업

```yaml
lifecycle:
  preRemove:
    - name: Backup workspace before removal
      command: |
        mkdir -p backups
        tar -czf backups/wiz-workspace-backup.tar.gz workspace
      timeout: 5m
```

중요: lifecycle은 호스트에서 실행됨. 따라서 이 백업은 오히려 적절한 lifecycle 사용 사례임. Sandbox 내부 DB 메모리 상태를 자동으로 일관성 있게 백업해주는 것은 아니므로 MariaDB는 mysqldump 같은 DB 전용 백업도 별도로 고려함.

27. 날짜가 들어간 백업 스크립트

```bash
#!/bin/bash
set -e

mkdir -p backups
STAMP=$(date +%Y%m%d_%H%M%S)

tar -czf "backups/workspace_${STAMP}.tar.gz" workspace

echo "Backup completed: backups/workspace_${STAMP}.tar.gz"
```

28. MariaDB 논리 백업

```bash
# Sandbox 내부에서 예
mariadb-dump -u devuser -p wizdb > /workspace/wizdb_backup.sql
```

DB dump를 workspace 아래에 두면 로컬 호스트에도 보존됨. 운영 데이터는 보안·암호화·접근권한을 별도로 고려해야 함.

29. Cloud persistent volume

Cloud Sandbox의 persistent volume은 로컬 workspace와 다른 기능임. Cloud-only이며 데이터는 Sandbox가 종료될 때 snapshot으로 저장됨. 여러 Sandbox가 같은 volume을 동시에 mount하면 마지막 종료 Sandbox의 snapshot이 이전 snapshot을 덮어쓸 수 있으므로 DB 동시 쓰기 용도로는 주의해야 함.

```bash
sbx --cloud volume create wiz-dbdata
sbx --cloud volume ls
sbx --cloud volume inspect wiz-dbdata

# Cloud Sandbox 생성 시 기존 volume 연결 예
sbx --cloud create --volume wiz-dbdata:/var/lib/mysql codex
```

30. Cloud volume과 로컬 workspace 비교

구분	로컬 workspace	Cloud persistent volume
용도	소스/프로젝트 파일 호스트 보존	Cloud Sandbox 간 데이터 보존
위치	호스트 디렉터리 bind mount	Docker Cloud volume
보존	호스트 파일이 그대로 남음	종료 시 snapshot
DB 사용	설계 가능하나 권한/성능 검증 필요	동시 mount 시 last-exit-wins 주의
명령	workspace: ./workspace	sbx --cloud volume ...

31. 최초 구축 순서

1. 프로젝트 루트 `wiz-sandbox` 생성
2. `workspace`, `docs`, `backups`, `scripts` 생성
3. `sbxenv.yaml` 작성
4. `sbx secret set openai --oauth` 실행
5. `sbx env plan`으로 변경 사항 확인
6. `sbx env create` 또는 `sbx env run`
7. 필요하면 Sandbox 내부 JDK/SSH/MariaDB 도구 준비
8. Spring Boot 프로젝트 생성/복사
9. MariaDB DB/사용자 초기화
10. `./gradlew clean build`
11. `./gradlew bootRun`
12. `sbx ports wiz-full-dev` 확인
13. LAN PC에서 2222/8080/3307 테스트

32. 매일 개발할 때의 정석 순서

```bash
cd wiz-sandbox

sbx env run --env-arg profile=dev

# 별도 터미널
sbx env exec -- ./gradlew clean build

# 테스트
sbx env exec -- ./gradlew test

# 실행
sbx env exec -- ./gradlew bootRun
```

33. Codex 중심 개발 루프

```text
1. Codex에게 요구사항 설명
2. Codex가 프로젝트 분석
3. Kotlin/Spring Boot 코드 수정
4. ./gradlew clean build
5. 테스트 실패 분석
6. 코드 재수정
7. ./gradlew test
8. Spring Boot 실행
9. REST API curl 테스트
10. git diff 검토
```

34. 상태 확인 명령

```bash
sbx version
sbx ls
sbx ports wiz-full-dev
sbx policy inspect wiz-full-dev

sbx env exec -- java -version
sbx env exec -- ./gradlew --version
sbx env exec -- bash -lc "ps aux"
sbx env exec -- bash -lc "ss -lntp"
```

35. SSH가 안 될 때

확인	명령/조치
Host 포트	Test-NetConnection HOST_IP -Port 2222
포트 게시	sbx ports wiz-full-dev
Sandbox 22 listen	sbx env exec -- bash -lc "ss -lntp | grep ':22 ' || true"
sshd 프로세스	sbx env exec -- bash -lc "ps aux | grep sshd"
사용자	Sandbox 내부 계정 존재 확인
인증	authorized_keys 또는 비밀번호 설정 확인

확인	명령/조치
방화벽	Docker Host OS 방화벽 2222 허용

36. Spring Boot 8080이 안 될 때

```bash
sbx env exec -- ./gradlew bootRun
sbx env exec -- bash -lc "ss -lntp | grep 8080 || true"
sbx env exec -- curl -v http://127.0.0.1:8080/
sbx ports wiz-full-dev
```

내부 curl은 되는데 외부에서 안 되면 port publish/host firewall을 우선 확인함. 내부 curl부터 안 되면 Spring Boot 기동 로그와 애플리케이션 오류를 먼저 확인함.

37. MariaDB 3307이 안 될 때

```bash
sbx env exec -- bash -lc "ss -lntp | grep 3306 || true"
sbx env exec -- mariadb -h 127.0.0.1 -P 3306 -u devuser -p
sbx ports wiz-full-dev
```

Access denied는 네트워크보다 사용자/비밀번호/host 권한 문제일 가능성이 큼. Connection refused는 MariaDB 미기동 또는 listen 주소/포트 문제를 우선 확인함.

38. Gradle build가 안 될 때

```bash
sbx env exec -- java -version
sbx env exec -- ./gradlew --version
sbx env exec -- ./gradlew clean build --stacktrace
```

- JDK 버전 확인
- gradlew 실행권한 확인
- 의존성 다운로드 네트워크 확인
- Kotlin/Spring Boot plugin 버전 확인
- MariaDB 연결이 테스트 단계에서 필요한지 확인

39. Codex 인증이 안 될 때

```bash
sbx secret ls
sbx secret set openai --oauth
sbx env rm
sbx env run
```

환경의 secrets/bindings 등 생성 시 적용되는 항목은 기존 Sandbox에 즉시 반영되지 않을 수 있으므로 환경 재생성이 필요한 경우가 있음.

40. 환경 변경 후 재생성해야 하는 경우

공식 환경 파일 동작상 workspace, kits, ports, secrets, bindings, sandboxOptions 등의 변경은 기존 Sandbox에 즉시 적용되지 않고 다음 생성 시 반영되는 항목이 있음. 이런 경우:

```bash
sbx env plan
sbx env rm
sbx env create
sbx env run
```

41. 보안 권장 구성

- Spring Boot 8080만 LAN 공개하고 SSH/MariaDB는 가능하면 loopback 유지.
- SSH는 비밀번호보다 public key 인증을 우선 고려.
- MariaDB 3307을 인터넷 공유기 포트포워딩으로 직접 공개하지 않음.
- OpenAI API 키를 sbxenv.yaml, application.yml, Git 저장소에 기록하지 않음.
- DB 비밀번호도 Git에 넣지 않음.
- docs read-only workspace를 사용해 중요한 원본 자료의 우발적 수정을 방지.
- 삭제 전 workspace 백업과 MariaDB dump를 별도로 수행.

42. 더 안전한 포트 설정 예

```yaml
ports:
  # SSH는 Docker Host에서만
  - sandbox: 22
    host: 2222
    hostIP: 127.0.0.1
    protocol: tcp4

  # Spring Boot만 LAN 공개
  - sandbox: 8080
    host: 8080
    hostIP: 0.0.0.0
    protocol: tcp4

  # MariaDB도 Host에서만
  - sandbox: 3306
    host: 3307
    hostIP: 127.0.0.1
    protocol: tcp4
```

실제 개발에서는 이 구성이 전체 0.0.0.0 공개보다 더 권장됨.

43. 완전체 실행 명령 모음

```bash
# 1. OpenAI 인증
sbx secret set openai --oauth

# 2. 환경 검토
sbx env plan

# 3. 개발 profile로 실행
sbx env run --env-arg profile=dev

# 4. Bash
sbx env exec -it -- bash

# 5. Java/Gradle
sbx env exec -- java -version
sbx env exec -- ./gradlew --version

# 6. Build
sbx env exec -- ./gradlew clean build

# 7. Test
sbx env exec -- ./gradlew test

# 8. Spring Boot
sbx env exec -- ./gradlew bootRun

# 9. 포트
sbx ports wiz-full-dev

# 10. 목록
sbx ls

# 11. 제거 전 계획
sbx env plan

# 12. 제거
sbx env rm
```

44. 초보자가 기억할 핵심 10개

No.	명령	의미
1	sbx env plan	YAML 적용 전에 반드시 확인
2	sbx env run	환경 적용 + Codex attach
3	sbx env exec -it -- bash	Sandbox shell
4	sbx env exec -- ./gradlew clean build	Spring Boot 빌드
5	sbx ports wiz-full-dev	포트 확인
6	sbx ls	Sandbox 목록
7	sbx secret set openai --oauth	Codex 인증
8	sbx env rm	환경 제거

No.	명령	의미
9	Test-NetConnection HOST -Port 8080	Windows 외부 포트 검사
10	ss -lntp	Sandbox 내부 listen 검사

45. Docker Compose와 sbxenv.yaml 비교

구분	docker-compose.yml	sbxenv.yaml
주 목적	여러 컨테이너 서비스 오케스트레이션	AI agent Sandbox 환경 선언
MariaDB service	services로 별도 컨테이너 선언 가능	Compose services 방식 아님
Agent	기본 개념 아님	agent: codex 등
Workspace	volume/bind mount	workspace/additionalWorkspaces
Port	ports	ports
CPU/RAM	deploy/resource 등 구성	sandboxOptions
Secret	Compose secret/env 등	Sandbox service secret/credentials
Lifecycle	entrypoint/command 등	호스트 lifecycle command

MariaDB를 Spring Boot와 완전히 분리된 별도 컨테이너로 안정적으로 운영하고 싶다면 Docker Compose가 더 자연스러운 구조일 수 있음. Docker Sandboxes는 Codex 같은 agent를 격리된 개발환경에서 작업시키는 데 초점이 있음.

46. 최종 권장 운영 방식

학습/개발 단계에서는 Docker Sandbox 안에 개발 도구와 MariaDB를 함께 두는 구성이 이해하기 쉬움. 그러나 장기 운영에서는 다음처럼 역할을 분리하는 것이 관리하기 쉬움.

```text
Docker Sandbox
├── Codex AI
├── JDK 21
├── Kotlin
├── Gradle
└── Spring Boot 개발/테스트

별도 DB 계층
└── MariaDB
    ├── Docker Compose 또는 전용 DB 서버
    ├── 독립 백업
    └── 접근제어
```

즉, Sandbox는 'AI 개발 워크스테이션', MariaDB는 '영구 데이터 서비스'로 분리하는 구조가 장기적으로 더 명확함. 다만 본 교본은 올인원 학습형 구성도 충분히 이해할 수 있도록 함께 설명함.

47. 공식 문서 기준 주의사항

- Docker Sandboxes 0.43.0은 2026-09-15 릴리스 기준이며 기능이 빠르게 변경되고 있음.
- `sbx env`와 `sbxenv.yaml`은 experimental 기능임.
- 프로젝트 환경 파일 이름은 현재 `sbxenv.yaml`이며 사용자 기본값은 `~/.sbxenv.yaml`임.

- 환경 plan은 host lifecycle command, credentials, bindings, MCP, workspace, kit, ports, sandbox options 등을 보여줌.
- workspace/kits/ports/secrets/bindings/sandboxOptions 변경은 기존 Sandbox를 재생성해야 반영되는 경우가 있음.
- Cloud persistent volume은 cloud-only이고 종료 시 snapshot 방식임.

48. 공식 참고자료

- Docker Docs - Docker Sandboxes release notes
- Docker Docs - Sandbox environment files
- Docker Docs - sbx CLI reference
- Docker Docs - sbx ports
- Docker Docs - sbx secret
- Docker Docs - sbx volume

49. 마지막 체크리스트

☐ sbxenv.yaml이 workspace 밖에 있는가?

☐ Codex agent가 지정되어 있는가?

☐ CPU 4 / RAM 8GB가 필요한가?

☐ OpenAI Secret을 평문으로 넣지 않았는가?

☐ SSH 2222 포트가 정말 LAN 공개가 필요한가?

☐ Spring Boot 8080 listen 상태가 정상인가?

☐ MariaDB 3306 listen 상태가 정상인가?

☐ Host 3307 포트를 인터넷에 공개하지 않았는가?

☐ workspace가 호스트에 실제로 존재하는가?

☐ docs가 readOnly로 연결되어 있는가?

☐ 삭제 전 backup이 만들어지는가?

☐ DB는 mysqldump 등 별도 백업이 있는가?

☐ 환경 변경 후 Sandbox 재생성이 필요한지 확인했는가?